

An aerial photograph of a large concrete dam with multiple spillways, situated in a lush green valley. A reservoir is formed upstream of the dam, and a road with a guardrail runs along the right side of the dam. The surrounding area is densely forested.

DP optimization model for reservoir operation

Team B 김현주 남현욱 이주형

Contents

- A 모델 설정
- B 프로그램 구축
- C 최적의 솔루션 분석
- D 유입량 차이에 따른 비교
- E 한계 및 극복방안
- F R Code



A 모델 설정

Project Understanding

- ❖ 저수지 운영에 DP가 필요한 이유?
 - 물 사용자는 유량에 대해 다양한 목표를 달성하고자 함
 - 하지만, 모든 요구사항을 충족하는 것은 불가능함
 - 따라서 운영자는 수요자에게 최적의 방법을 제시할 필요가 있음
- ❖ 문제 설명
 - 최적의 저수지 운영 을 위해 다년 네트워크를 통해 경로를 찾는 것
 - 다단계 의사 결정 문제



Set up

❖ 지배방정식

$$S_{t+1} = S_t + Q_t - R_t - L_t(S_t, S_{t+1}) \text{ for each period } t$$

Where

S_{t+1} = Final storage volume

S_t = Initial active storage volume

Q_t = Mean inflow

$L_t(S_t, S_{t+1})$ = Evaporation and seepage Losses

R_t = Release or discharge

*이 때 각 변수는 기간 t에 대한 부피 단위로 표시함

Set up

❖ 제약조건

- 저류조의 부피
 - 0~20MCM의 범위를 가짐
- 초기저류량(S_t)
 - 모든 season t 에 대해 0, 5, 10, 15, 20의 이산값을 가짐
- 초과저류량(ES_t)
 - $S_t \leq TF + ES_t$ for period $t = 1$ and 2
- 부족방류량(DR_t)
 - $R_t \geq TR_t - DR_t$ for all periods t
- 각 변수
 - 정수 (losses는 반올림)

period or season t	storage targets TS^R_t, TS^F_t	release target TR_t
1	15 flood control	≥ 10
2	20 recreation	≥ 15
3	20 recreation	≥ 20
4		≥ 15

E020820p

Set up

❖ Objective function

Minimize $\sum_t \mathbf{TSD}_t$

Where

Total Sum of squared Deviations:

$$\mathbf{TSD}_t = \mathbf{ws}_t[(\mathbf{TS} - \mathbf{S}_t)^2 + (\mathbf{TS} - \mathbf{S}_{t+1})^2] + \mathbf{wf}_t[(\mathbf{ES}_t)^2 + (\mathbf{ES}_{t+1})^2] + \mathbf{wr}_t[\mathbf{DR}_t^2]$$

$\mathbf{TS}$ = Target storage

$\mathbf{S}_t$ = Initial storage volume

$\mathbf{S}_{t+1}$ = Final storage volume

$\mathbf{ES}_t$ = the storage volume in excess of the flood storage target volume

$\mathbf{DR}_t$ = difference between the actual release, $\mathbf{R}_t$, and the target release, $\mathbf{TR}_t$, when the release is less than the target.

$\mathbf{ws}_t$ = The weights associated with recreation. (period 2, 3 =1, period 1, 4 = 0)

$\mathbf{wf}_t$ = The weights associated with flood control. (period 1 =1, period 2, 3, 4 = 0)

$\mathbf{wr}_t$ = The weights associated with release. (All periods =1)

Set up

❖ Objective function

- s_t 와 s_{t+1} 에 따른 계절별 TSD 결과

$TR_1 \geq 10, TS_t^F \leq 15$
period $t = 1$

initial storage	final storage				
	0	5	10	15	20
0	0	0	0	4	74
5	0	0	0	0	29
10	0	0	0	0	25
15	0	0	0	0	25
20	25	25	25	25	50

TDS_1

$TS_2 = 20, TR_2 \geq 15$
period $t = 2$

0	809	689	696	1	1
5	625	466	406	446	1
10	500	325	216	206	296
15	425	250	125	66	125
20	400	225	100	25	25

TDS_2

$TS_3 = 20, TR_3 \geq 20$
period $t = 3$

0	996	1025	--	---	--
5	725	675	725	---	--
10	525	425	425	525	--
15	425	275	225	306	--
20	400	225	136	146	256

TDS_3

$TR_4 \geq 15$
period $t = 4$

0	0	4	49	144	---
5	0	0	4	49	144
10	0	0	0	4	64
15	0	0	0	0	9
20	0	0	0	0	0

TDS_4

E020820q

Set up

❖ Recursive equations (시작 단계)

$$F_t^n(S_t, Q_t) = \underset{R_t}{\text{minimum}} \sum_{t=1,n} \text{TSD}_t(S_t, R_t, S_{t+1}) \quad \text{over all feasible values of } R_t$$

Where,

$$S_{t+1} = S_t + Q_t - R_t - L_t(S_t, S_{t+1})$$

And,

$S_t < K$, the capacity of the reservoir

Set up

❖ Recursive equations

$$F_t^n(S_t, Q_t) = \text{minimum} \left[\sum_{t=1, n} TSD_t(S_t, R_t, S_{t+1}) + F_{t+1}^{n-1}(S_{t+1}, Q_{t+1}) \right] \text{ for all } 0 \leq S_t \leq 20$$

➡ 찾고자 하는 정책에 도달할 때 까지 계산을 반복 수행

▪ 결정변수가 방류량(R_t)인 경우의 제약조건

$$R_t \leq S_t + Q_t - L_t(S_t, S_{t+1})$$

$$R_t \geq S_t + Q_t - L_t(S_t, S_{t+1}) - 20 \text{ (the capacity)}$$

And

$$S_{t+1} = S_t + Q_t - R_t - L_t(S_t, S_{t+1})$$

▪ 결정변수가 최종저류량(S_{t+1})인 경우의 제약조건

$$0 \leq S_{t+1} \leq 20$$

$$S_{t+1} \leq S_t + Q_t - R_t - L_t(S_t, S_{t+1})$$

And

$$R_t = S_t + Q_t - L_t(S_t, S_{t+1})$$

Set up

❖ Assumption

- Recreation, flood control, release를 고려한 **weights**의 값이 극단적임
- 모델에서 설정한 손실(증발산, 누수)외의 손실값은 고려하지 않음
- 계산되는 모든 값이 정수로 계산되어야 함
- 미래 유입량 및 용수 수요량의 변동성을 고려하지 않음
- 결정된 최적 정책은 변하지 않음
- Net benefit은 state, decision variable에 의해서만 발생함
- 각 단계는 독립적임
- 설계단계와 운영 룰을 둘 다 모르는 경우 문제를 풀 수 없음

Ultimate Objective

- ❖ 'Steady-state' 저수지 운영 정책을 찾는 것
 - R_t 와 S_{t+1} 이 각각 모든 계절에 대해 같은 경우

Same!

period $t = 2$ $n = 3$ $Q_2 = 12$

$$F_2^3(S_2, Q_2) = \min \{TSD_2(S_2, R_2, S_3) + F_3^2(S_3, Q_3)\}$$

initial storage S_2	$TSD_2 + F_3^2(S_3, Q_3)$					F_2^3 (S_2, Q_2)	R_2	S_3
	0	5	10	15	20			
0	809+996	689+675	696+425	-	-	1121	1	10
5	625+996	466+675	406+425	446+225	-	671	1	15
10	500+996	325+675	216+425	206+225	296+136	431	6	15
15	425+996	250+675	125+425	66+225	125+136	261	5	20
20	400+996	225+675	100+425	25+225	25+136	161	10	20

period $t = 2$ $n = 7$ $Q_2 = 12$

$$F_2^7(S_2, Q_2) = \min \{TSD_2(S_2, R_2, S_3) + F_3^6(S_3, Q_3)\}$$

initial storage S_2	$TSD_2 + F_3^6(S_3, Q_3)$					F_2^7 (S_2, Q_2)	R_2	S_3
	0	5	10	15	20			
0	809+1190	689+865	696+611	-	-	1307	1	10
5	625+1190	466+865	406+611	446+411	-	857	1	15
10	500+1190	325+865	216+611	206+411	296+322	617	6	15
15	425+1190	250+865	125+611	66+411	125+322	447	5	20
20	400+1190	225+865	100+611	25+411	25+322	347	10	20

Ultimate Objective

❖ 'Steady-state' 저수지 운영 정책을 찾는 것

- $F_t^n - F_t^{n-4}$ 이 같아지는 경우

period $t = 2$ $n = 3$ $Q_2 = 12$

$$F_2^3(S_2, Q_2) = \min \{TSD_2(S_2, R_2, S_3) + F_3^2(S_3, Q_3)\}$$

initial storage S_2	$TSD_2 + F_3^2(S_3, Q_3)$					$F_2^3(S_2, Q_2)$	R_2	S_3
	0	5	10	15	20			
0	809+996	689+675	696+425	-	-	1121	1	10
5	625+996	466+675	406+425	446+225	-	671	1	15
10	500+996	325+675	216+425	206+225	296+136	431	6	15
15	425+996	250+675	125+425	66+225	125+136	261	5	20
20	400+996	225+675	100+425	25+225	25+136	161	10	20

period $t = 2$ $n = 7$ $Q_2 = 12$

$$F_2^7(S_2, Q_2) = \min \{TSD_2(S_2, R_2, S_3) + F_3^6(S_3, Q_3)\}$$

initial storage S_2	$TSD_2 + F_3^6(S_3, Q_3)$					$F_2^7(S_2, Q_2)$	R_2	S_3
	0	5	10	15	20			
0	809+1190	689+865	696+611	-	-	1307	1	10
5	625+1190	466+865	406+611	446+411	-	857	1	15
10	500+1190	325+865	216+611	206+411	296+322	617	6	15
15	425+1190	250+865	125+611	66+411	125+322	447	5	20
20	400+1190	225+865	100+611	25+411	25+322	347	10	20

Initial storage S	$F_t^n - F_t^{n-4}$
0	186
5	186
10	186
15	186
20	186

Minus

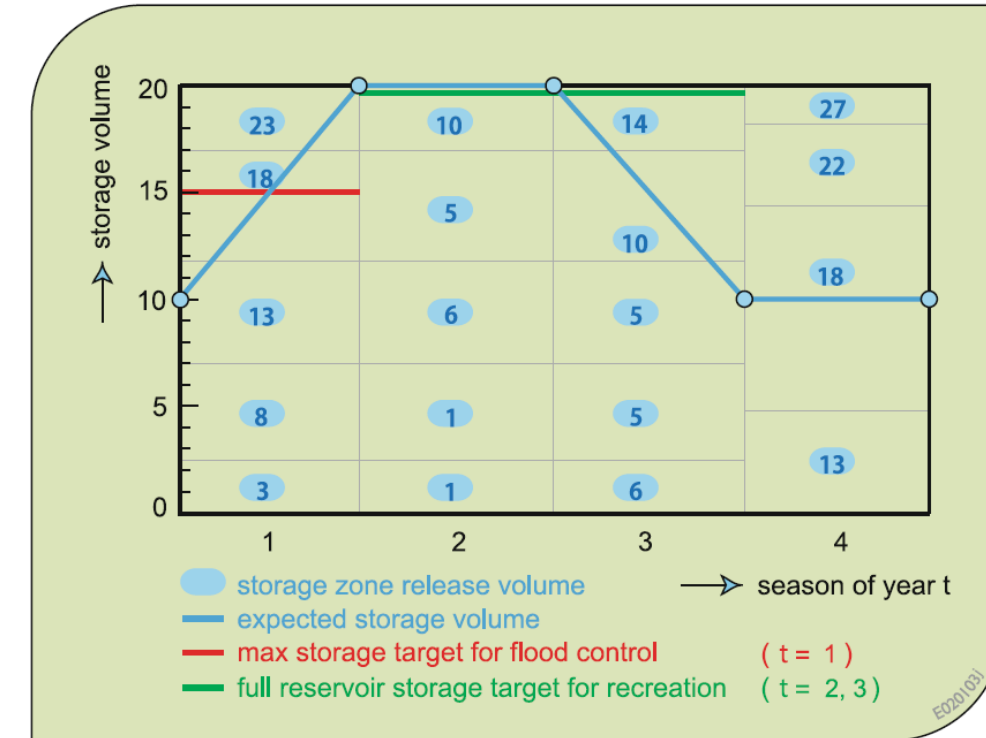
Same!

➡ 위의 두 가지 조건을 만족하는 경우 steady-state 정책에 도달했다고 봄

Ultimate Objective

❖ Steady-state 저수지 운영 정책 & Rule curve

initial storage S	release season t				final storage volume season t			
	1	2	3	4	1	2	3	4
0	3	1	6	13	20	10	0	5
5	8	1	5	13-18	20	15	5	5-10
10	13	6	5	18	20	15	10	10
15	18	5	10	17-23	20	20	10	10-15
20	23	10	14	22-27	20	20	10	10-15



Ultimate Objective

- ❖ 앞서 정의한 'Steady-state' 저수지 운영 정책에 도달한 경우 최적이라고 할 수 있음
 - 공급자의 입장을 고려하여 저수량의 범위, 목표 방류량, 홍수기 목표 저수량 등이 constraints로 포함되어 있음
 - 수요자의 입장을 고려하여 홍수방어, 수상레저, 방류조건이 weights로 포함되어 있음

❖ "Optimality"



공급자 입장	수요자 입장
• 댐 운영의 효율성 • 댐 운영 룰에 의한 목표 방류량 정량화	• 범람 피해를 입지 않는 것 • 생활, 농업, 공업에 있어 용수공급 부족을 겪지 않는 것 • 수상레저를 즐길 수 있는 충분한 물의 양

B 모델 구축

Program built

1. DP 돌리기 위한 변수 입력

(Volume units for the period t)

- Discrete state variable
 - 5 Storage volumes: $s \leftarrow c(0, 5, 10, 15, 20)$
 - Mean flow: $\text{inflow} \leftarrow c(24, 12, 6, 18)$
- Evaporation and seepage losses according to initial and final storage of each season:
 $\text{loss} \leftarrow \text{list}(\text{loss.1}, \text{loss.2}, \text{loss.3}, \text{loss.4})$
- Release $R_t = S_t + Q_t - L_t(S_t, S_{t+1}) - S_{t+1}$ for each period t:
 $\text{Release} \leftarrow \text{list}(\text{release.1}, \text{release.2}, \text{release.3}, \text{release.4})$

2. TSD 설정

- Sum of total weighted squared deviations, TSD_t i.e. value of benefit function
 - tds.weight
 - tds.conditions

```
> tds.weight
```

	[,1]	[,2]	[,3]
Season t [1,]	0	1	1
[2,]	1	0	1
[3,]	1	0	1
[4,]	0	0	1
	ws_t	wf_t	wr_t

```
> tds.conditions
```

	[,1]	[,2]	[,3]
Season t [1,]	15	15	10
[2,]	20	20	15
[3,]	20	20	20
[4,]	0	0	15
	TS_t^R	TS_t^F	TR_t

Storage targets

Release targets

Program built

- TSD_t
 $tds = (tds.w[1] * wst) + (tds.w[2] * wft) + (tds.w[3] * wrt)$
 $ft.dat[j,k] = tds$
 $ft \leftarrow list(ft.1, ft.2, ft.3, ft.4)$

3. 역방향 재귀 진행

- Reordered list of release and ft to conduct backward recursive algorithm
 $back.release$ and $back.ft$
- Recursive equation
 $ft.model[k,l] = ft.model[k,l] + back.step.result[,1][[l]]$
 $ft.r = data.frame(cbind(ft.l), cbind(release.l), cbind(storage.l))$

4. 첫 번째 단계($t=4$)에 대해 진행 후 역방향 재귀 알고리즘 실행

- Combined result that classified by each season
 $ft.result$

Program built

5. 이산 steady-state 저수지 운영 정책 찾기

- if ($l > 4$) {
 - $a = \text{unlist}(\text{ft.result}[j][[l-1]][1]) - \text{unlist}(\text{ft.result}[j][l][1])$
 - $b = \text{unlist}(\text{ft.result}[j][l][1]) - \text{unlist}(\text{ft.result}[j][[l+1]][1])$
 - if ($a[1] == b[1] \ \& \ a[2] == b[2] \ \& \ a[3] == b[3] \ \& \ a[4] == b[4]$) break

결과

```
> ft.result
[[1]]
[[1]][[1]]
ft.l release.l storage.l
1 0 18 0
2 0 23, 18 0, 5
3 0 28, 23, 18 0, 5, 10
4 0 33, 28, 23, 17 0, 5, 10, 15
5 0 38, 33, 27, 22, 17 0, 5, 10, 15, 20

[[1]][[2]]
ft.l release.l storage.l
1 194 13 5
2 190 18, 13 5, 10
3 186 18 10
4 186 23, 17 10, 15
5 186 27, 22 10, 15

[[1]][[3]]
ft.l release.l storage.l
1 380 13 5
2 376 18, 13 5, 10
3 372 18 10
4 372 23, 17 10, 15
5 372 27, 22 10, 15

[[1]][[4]]
ft.l release.l storage.l
1 566 13 5
2 562 18, 13 5, 10
3 558 18 10
4 558 23, 17 10, 15
5 558 27, 22 10, 15
```

```
[[2]]
[[2]][[1]]
ft.l release.l storage.l
1 996 6 0
2 675 5 5
3 425 10, 5 5, 10
4 225 10 10
5 136 14 10

[[2]][[2]]
ft.l release.l storage.l
1 1190 6 0
2 865 5 5
3 611 5 10
4 411 10 10
5 322 14 10

[[2]][[3]]
ft.l release.l storage.l
1 1376 6 0
2 1051 5 5
3 797 5 10
4 597 10 10
5 508 14 10
```

```
[[3]]
[[3]][[1]]
ft.l release.l storage.l
1 1121 1 10
2 671 1 15
3 431 6 15
4 261 5 20
5 161 10 20

[[3]][[2]]
ft.l release.l storage.l
1 1307 1 10
2 857 1 15
3 617 6 15
4 447 5 20
5 347 10 20

[[3]][[3]]
ft.l release.l storage.l
1 1493 1 10
2 1043 1 15
3 803 6 15
4 633 5 20
5 533 10 20
```

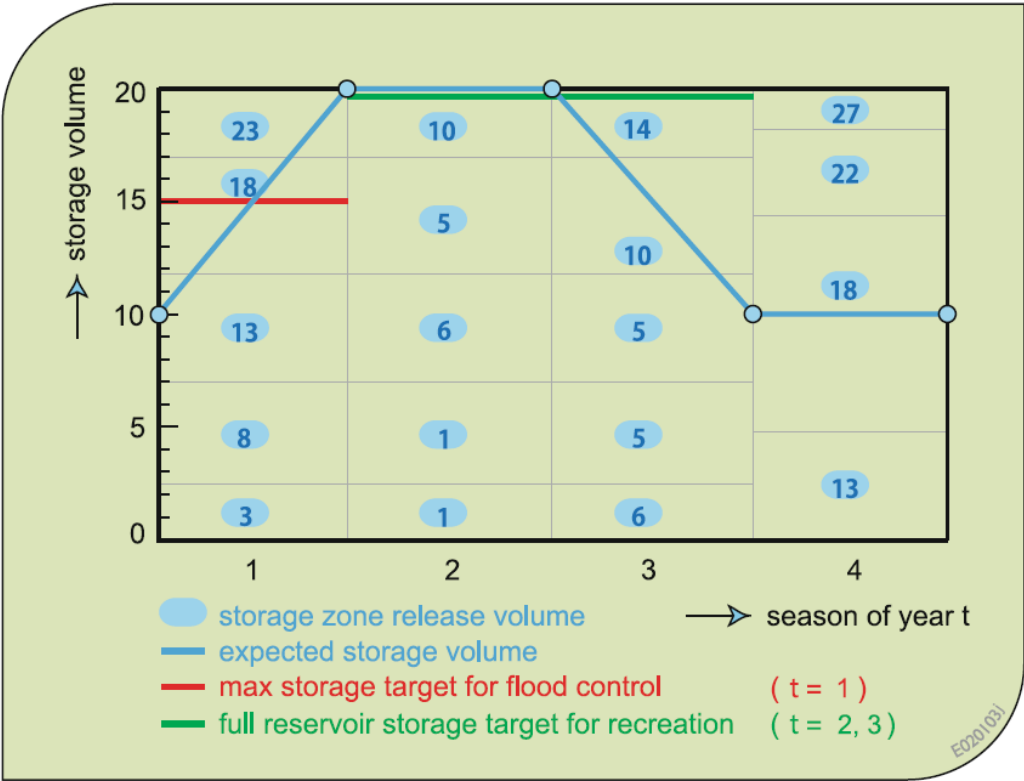
```
[[4]]
[[4]][[1]]
ft.l release.l storage.l
1 235 3 20
2 190 8 20
3 186 13 20
4 186 18 20
5 211 23 20

[[4]][[2]]
ft.l release.l storage.l
1 421 3 20
2 376 8 20
3 372 13 20
4 372 18 20
5 397 23 20

[[4]][[3]]
ft.l release.l storage.l
1 607 3 20
2 562 8 20
3 558 13 20
4 558 18 20
5 583 23 20
```

Results

❖ 이산 steady-state 저수지 운영 정책 결과



Initial storage S	Release Season t			
	1	2	3	4
0	3	1	6	13
5	8	1	5	13-18
10	13	6	5	18
15	18	5	10	17-23
20	23	10	14	22-27

Initial storage S	Final storage volume Season t			
	1	2	3	4
0	20	10	0	5
5	20	15	5	5-10
10	20	15	10	10
15	20	20	10	10-15
20	20	20	10	10-15



C 최적의 솔루션 분석

Steady-state policy

- ❖Optimal solution
 - R_t 와 S_{t+1} 이 각각 모든 계절에 대해 같은 경우
 - $F_t^n - F_t^{n-4}$ 이 같아지는 경우

Season t	시행횟수 n
1	4, 8
2	3, 7
3	6, 10
4	5, 9

Initial storage S	$F_t^n - F_t^{n-4}$
0	186
5	186
10	186
15	186
20	186

period t = 2 n = 3 Q₂ = 12

$F_2^3(S_2, Q_2) = \min \{TSD_2(S_2, R_2, S_3) + F_3^2(S_3, Q_3)\}$

initial storage S ₂	TSD ₂ + F ₃ ² (S ₃ , Q ₃)					F ₂ ³ (S ₂ , Q ₂)	R ₂	S ₃
	0	5	10	15	20			
0	809+996	689+675	696+425	-	-	1121	1	10
5	625+996	466+675	406+425	446+225	-	671	1	15
10	500+996	325+675	216+425	206+225	296+136	431	6	15
15	425+996	250+675	125+425	66+225	125+136	261	5	20
20	400+996	225+675	100+425	25+225	25+136	161	10	20

period t = 2 n = 7 Q₂ = 12

$F_2^7(S_2, Q_2) = \min \{TSD_2(S_2, R_2, S_3) + F_3^6(S_3, Q_3)\}$

initial storage S ₂	TSD ₂ + F ₃ ⁶ (S ₃ , Q ₃)					F ₂ ⁷ (S ₂ , Q ₂)	R ₂	S ₃
	0	5	10	15	20			
0	809+1190	689+865	696+611	-	-	1307	1	10
5	625+1190	466+865	406+611	446+411	-	857	1	15
10	500+1190	325+865	216+611	206+411	296+322	617	6	15
15	425+1190	250+865	125+611	66+411	125+322	447	5	20
20	400+1190	225+865	100+611	25+411	25+322	347	10	20

Same!

Minus

Same!



Steady-state policy

❖결과 부분이 모든 상황에서 optimal한가? **NO!**

- 유입량이나 수요량의 유동성 고려 불가
- 가중치 적용 조건의 이분화 (flood는 시즌 1에만 발생하고, recreation은 시즌 2, 3에만 사용)
- 관로 설비 노후화 및 기후변화 등으로 인한 손실의 변동 고려 불가

❖Optimal solution의 가치

- 가치 있는 경우
 - 사계절이 뚜렷한 경우
 - 유입량/물 수요량이 예측한 수준으로 발생하는 경우(관측값이 stationary할 경우)
- 가치 있지 않은 경우
 - 예상한 계절 외의 홍수 발생 또는 recreation 수요가 증가할 경우
 - 유역 내 도시화로 인해 생활용수 등의 수요가 급증하는 경우
 - 가뭄으로 인한 농업용수 수요가 증가하는 경우

D 유입량 차이에 따른 비교

Unexpected inflow scenario

❖유입량 차이에 따른 비교

▪ Q1_t=24, 12, 6, 18인 경우

year	season t	St + Qt - Rt - Lt				St+1	TSDt
1	1	20	24	23	1	20	50
1	2	20	12	10	2	20	25
1	3	20	6	14	2	10	136
1	4	10	18	18	0	10	0
2	1	10	24	13	1	20	25
2	2	20	12	10	2	20	25
2	3	20	6	14	2	10	136
2	4	10	18	18	0	10	0
3	1	10	24	13	1	20	25
3	2	20	12	10	2	20	25
3	3	20	6	14	2	10	136
3	4	10	18	18	0	10	0
4	1	10	24	13	1	20	25
4	2	20	12	10	2	20	25
4	3	20	6	14	2	10	136
4	4	10	18	18	0	10	0
total sum of squared deviations, TSD, =							186

▪ Q2_t=6, 12, 24, 18인 경우

year	season t	St + Qt - Rt - Lt				St+1	TSDt
1	1	5	6	1	0	10	4
1	2	10	12	6	1	15	50
1	3	15	24	17	2	20	125
1	4	20	18	17	1	20	75
2	1	20	6	5	1	20	0
2	2	20	12	10	2	20	0
2	3	20	24	22	2	20	25
2	4	20	18	17	1	20	75
3	1	20	6	5	1	20	0
3	2	20	12	10	2	20	0
3	3	20	24	22	2	20	25
3	4	20	18	17	1	20	75
4	1	20	6	5	1	20	0
4	2	20	12	10	2	20	0
4	3	20	24	22	2	20	25
4	4	20	18	17	1	20	75
total sum of squared deviations, TSD, =							100



Unexpected inflow scenario

❖유입량 차이에 따른 비교

▪ Q1_t=24, 12, 6, 18인 경우

Initial storage S	Release Season t				Final storage volume Season t			
	1	2	3	4	1	2	3	4
0	3	1	6	13	20	10	0	5
5	8	1	5	13-18	20	15	5	5-10
10	13	6	5	18	20	15	10	10
15	18	5	10	17-23	20	20	10	10-15
20	23	10	14	22-27	20	20	10	10-15

Season t	시행횟수 n
1	4, 8
2	3, 7
3	6, 10
4	5, 9

$F_t^n - F_t^{n-4} = 186$

▪ Q2_t=6, 12, 24, 18인 경우

Initial storage S	Release Season t				Final storage volume Season t			
	1	2	3	4	1	2	3	4
0	1	1	13-18	8	5	10	10-15	10
5	1	1	13	8	10	15	15	15
10	0	6	18	13	15	15	15	15
15	5	11	17	12	15	15	20	20
20	5	10	22	17	20	20	20	20

Season t	시행횟수 n
1	4, 8
2	3, 7
3	6, 10
4	5, 9

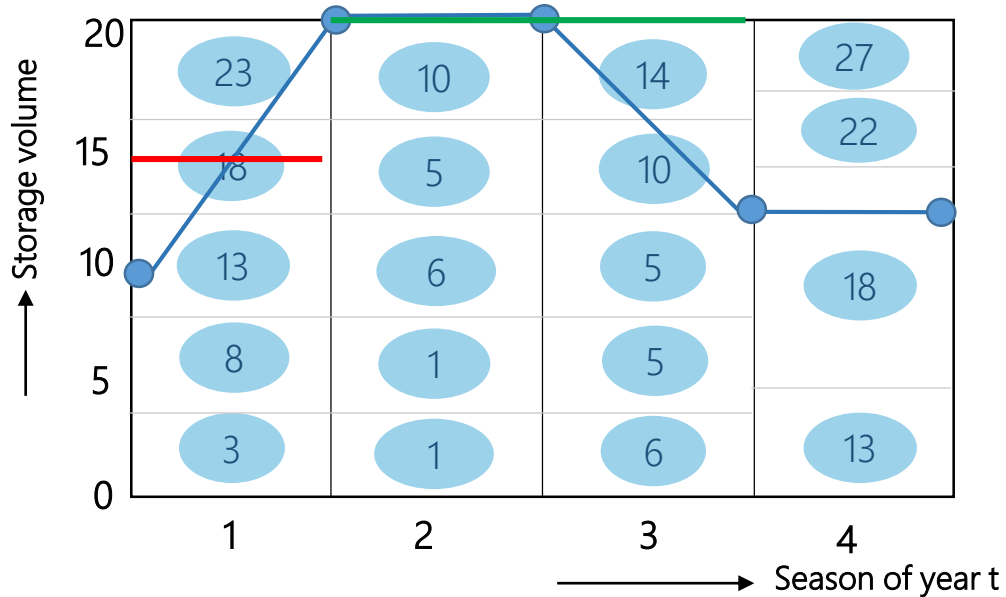
$F_t^n - F_t^{n-4} = 100$



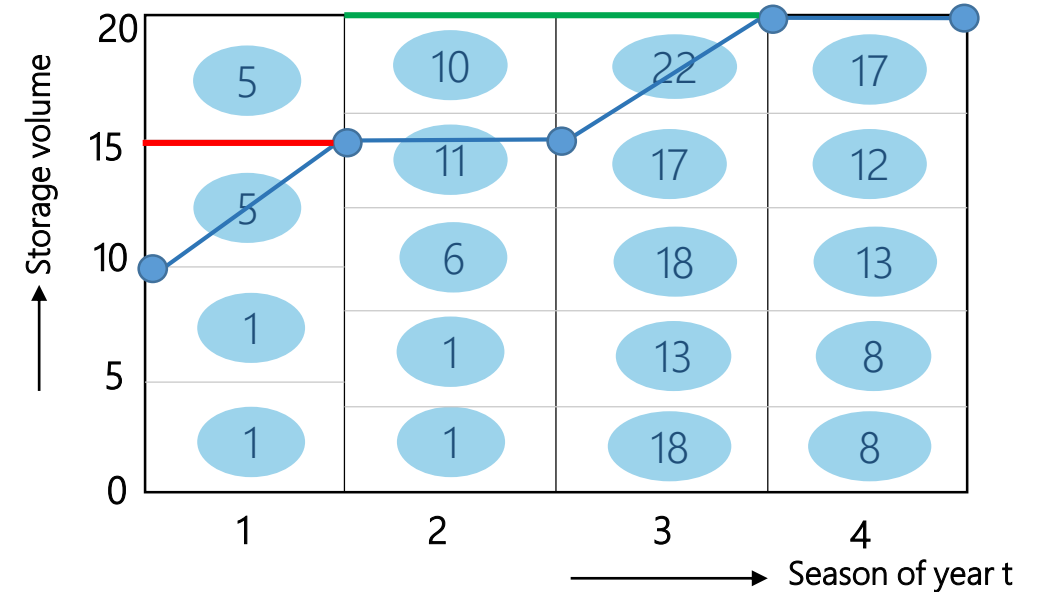
Unexpected inflow scenario

❖ 예상치 못한 유입량이 유입될 경우

- $Q1_t=24, 12, 6, 18$ 일 때 최적의 경로



- When $Q2_t=6, 12, 24, 18$ 일 때 최적의 경로



- Storage zone release volume (blue oval)
- Expected storage volume (blue line)
- Max storage target for flood control ($t=1$) (red line)
- Full reservoir storage target for recreation ($t=2, 3$) (green line)

- 'Steady-state' 저수지 운영 정책이 결정되는 n (시행횟수)은 같으나, $(F_t^n - F_t^{n-4})$ 의 값이 달라짐
- 각 시즌별 방류량, 최종저류량의 값이 달라짐
- 유입량의 변화가 극값으로 크거나 작을 경우 시행횟수가 길어질수도 있을 것이라 판단됨

E 한계 및 극복방안

Limitations

- ❖ 초기 설정 조건(유입량, 저수량 등)이 바뀌게 될 경우 TSD값이 변하여 'Steady-state' 저수지 운영 정책이 변하게 됨
- ❖ 이산화 때문에 계산횟수가 기하급수적으로 커짐 (Curse of dimensionality)
- ❖ 모든 정보는 모델 안에 상태변수로 포함되어 있어야 함 (Curse of modeling)
- ❖ Weights에 대한 신뢰성이 떨어짐 (Curse of multiple objectives)
 - Ex. Flood control 함수의 weight는 season 1에만 포함됨
- ❖ 최적의 경로라 하더라도 해당 경로가 발생할 확률에 대한 가중치가 포함되지 않음

Improvements

- ❖ Season별로 Inflow를 변수로 만들어 계산
- ❖ Weights의 신뢰성을 높일 방안을 강구
 - Ex. Flood control에 대한 weight를 모든 시즌에 대하여 적게라도 부여
- ❖ 확률 가중치를 포함하여 계산
- ❖ 개선이 가능한 부분도 있으나, 모델의 특성상 개선이 불가능한 부분(수치의 이산화, 많은 계산량 등)이 있음

F R Code

```

1 library(daewr)
2
3 #Term project A
4
5 #Storage
6 s = c(0, 5, 10, 15, 20)
7
8 #Loss
9 loss.1 = matrix(c(0, 0.1, 0.3, 0.4, 0.5,
10                  0.1, 0.2, 0.4, 0.6, 0.8,
11                  0.3, 0.4, 0.6, 0.8, 1.0,
12                  0.4, 0.6, 0.8, 1.0, 1.2,
13                  0.6, 0.8, 1.0, 1.2, 1.4), nrow=5, byrow=T)
14
15 loss.2 = matrix(c(0, 0.5, 0.7, 0.8, 1.0,
16                  0.5, 0.7, 0.8, 1.0, 1.2,
17                  0.7, 0.8, 1.0, 1.2, 1.4,
18                  0.8, 1.0, 1.2, 1.4, 1.6,
19                  1.0, 1.2, 1.4, 1.6, 1.8), nrow=5, byrow=T)
20
21 loss.3 = matrix(c(0, 0.7, 0.9, 1.0, 1.2,
22                  0.7, 0.9, 1.0, 1.2, 1.4,
23                  0.9, 1.0, 1.2, 1.4, 1.6,
24                  1.0, 1.2, 1.4, 1.6, 1.8,
25                  1.2, 1.4, 1.6, 1.8, 2.0), nrow=5, byrow=T)
26
27 loss.4 = matrix(c(0, 0.1, 0.2, 0.3, 0.4,
28                  0.1, 0.2, 0.3, 0.4, 0.5,
29                  0.2, 0.3, 0.4, 0.5, 0.6,
30                  0.3, 0.4, 0.5, 0.6, 0.7,
31                  0.4, 0.5, 0.6, 0.7, 0.8), nrow=5, byrow=T)
32
33 loss = list(loss.1, loss.2, loss.3, loss.4)
34
35 #Release
36 inflow = c(24, 12, 6, 18)
37
38 release.1 = array(NA, c(5,5))
39 release.2 = array(NA, c(5,5))
40 release.3 = array(NA, c(5,5))
41 release.4 = array(NA, c(5,5))
42
43 release = list(release.1, release.2, release.3, release.4)
44
45 for (i in 1:4){
46   #i=1
47   l = loss[[i]]
48   r = release[[i]]
49   q = inflow[i]
50

```

```

51   for (j in 1:5){
52     #j=1
53     for (k in 1:5){
54       #k=1
55       ll = l[j,k]
56       ll = round(ll)
57       rr = q + s[j] - s[k] - ll
58       if (rr < 0){
59         r[j,k] = NA
60         next
61       }
62       r[j,k] = round(rr)
63     }
64   }
65   release[[i]] = r
66 }
67
68 release[[1]][1,4] = 8
69 release[[1]][1,5] = 3
70 release[[1]][2,3] = 18
71
72 #ft
73 ft.1 = array(NA, c(5,5))
74 ft.2 = array(NA, c(5,5))
75 ft.3 = array(NA, c(5,5))
76 ft.4 = array(NA, c(5,5))
77 ft = list(ft.1, ft.2, ft.3, ft.4)
78
79 tds.weight = matrix(c(0,1,1,
80                      1,0,1,
81                      1,0,1,
82                      0,0,1), nrow=4, byrow = T)
83
84 tds.conditions = matrix(c(15,15,10,
85                          20,20,15,
86                          20,20,20,
87                          0,0,15), nrow=4, byrow=T)
88
89
90

```



```

91 for (i in 1:4){
92   #i=2
93   release.dat = release[[i]]
94   ft.dat = ft[[i]]
95   tds.w = tds.weight[i,]
96   tds.c = tds.conditions[i,]
97
98   for (j in 1:5){
99     #j=4
100    for (k in 1:5){
101      #k=5
102      if (is.na(release.dat[j,k])==TRUE){
103        ft.dat[j,k] = NA
104        next
105      }
106
107      wst = ((s[j]-tds.c[1])^2) + ((s[k]-tds.c[1])^2)
108
109      if (s[j] > tds.c[2] & s[k] <= tds.c[2]){
110        wft = (s[j] - tds.c[2])^2
111      }else if (s[k] > tds.c[2] & s[j] <= tds.c[2]){
112        wft = (s[k] - tds.c[2])^2
113      }else if (s[k] > tds.c[2] & s[j] > tds.c[2]){
114        wft = ((s[j] - tds.c[2])^2) + ((s[k] - tds.c[2])^2)
115      }else{
116        wft = 0
117      }
118
119      if (release.dat[j,k] < tds.c[3]){
120        wrt = (release.dat[j,k] - tds.c[3])^2
121      }else{
122        wrt = 0
123      }
124
125      tds = (tds.w[1] * wst) + (tds.w[2] * wft) + (tds.w[3] * wrt)
126      ft.dat[j,k] = tds
127    }
128  }
129  ft[[i]] = ft.dat
130 }

```

```

133 #Backward recursive process
134 back.ft =list(ft[[4]], ft[[3]], ft[[2]], ft[[1]])
135 back.release = list(release[[4]], release[[3]], release[[2]], release[[1]])
136 ft.result = list(c(),c(),c(),c())
137
138 #First step (T=3)
139 ft.model = back.ft[[1]]
140 back.r = back.release[[1]]
141 ll = length(ft.result[[1]])
142 ft.l = list()
143 release.l = list()
144 storage.l = list()
145 for(k in 1:5){
146     #k=4
147     if (is.na(ft.model[k,1]) & is.na(ft.model[k,2]) & is.na(ft.model[k,3]) & is.na(ft.model[k,4])){
148         next
149     }
150     s0 = min(ft.model[k,],na.rm=T)
151     s0.loc = which(ft.model[k,]==s0)
152     ss = s[s0.loc]
153     r = back.r[k,s0.loc]
154
155     ft.l[[k]] = s0
156     release.l[[k]] = c(r)
157     storage.l[[k]] = c(ss)
158
159 }
160
161 ft.r = data.frame(cbind(ft.l), cbind(release.l), cbind(storage.l))
162
163 ft.result[[1]][[1]] = ft.r

```

```

165 #Backward recursive algorithm
166 for (i in 2:100){
167     #i=7
168     j = mod(i,4)
169     bb = mod((i-1),4)
170
171     if (j == 0){
172         j = j+4
173     }
174
175     if (bb == 0){
176         bb = bb+4
177     }
178
179     ft.model = back.ft[[j]]
180     back.r = back.release[[j]]
181     ll = length(ft.result[[j]])
182     lll = length(ft.result[[bb]])
183
184     back.step.result = ft.result[[bb]][[lll]]
185
186     ft.l = list()
187     release.l = list()
188     storage.l = list()
189     for (k in 1:5){
190         #k=1
191         if (is.na(ft.model[k,1]) & is.na(ft.model[k,2]) & is.na(ft.model[k,3]) & is.na(ft.model[k,4]) & is.na(ft.model[k,5])){
192             ft.l[[k]] = NA
193             release.l[[k]] = NA
194             storage.l[[k]] = NA
195             next
196         }
197
198         for (l in 1:5){
199             #l=2
200             if (is.na(ft.model[1,l]) & is.na(ft.model[2,l]) & is.na(ft.model[3,l]) & is.na(ft.model[4,l]) & is.na(ft.model[5,l])){
201                 next
202             }
203             if (is.na(ft.model[k,l])){
204                 next
205             }
206             if (is.na(back.step.result[,l][[lll]])){
207                 ft.model[k,l] = NA
208                 next
209             }
210             ft.model[k,l] = ft.model[k,l] + back.step.result[,l][[lll]]
211         }
212     }

```

```

213
214     s0 = min(ft.model[k,],na.rm=T)
215     s0.loc = which(ft.model[k,]==s0)
216     ss = s[s0.loc]
217     r = back.r[k,s0.loc]
218
219     ft.l[[k]] = s0
220     release.l[[k]] = c(r)
221     storage.l[[k]] = c(ss)
222
223 ^ }
224
225 ft.r = data.frame(cbind(ft.l), cbind(release.l), cbind(storage.l))
226
227 ft.result[[j]][[(ll+1)]] = ft.r
228
229 ^ if (ll>4){
230     a = unlist(ft.result[[j]][[(ll-1)]][,1]) - unlist(ft.result[[j]][[ll]][,1])
231     b = unlist(ft.result[[j]][[ll]][,1]) - unlist(ft.result[[j]][[(ll+1)]][,1])
232     if (a[1]==b[1] & a[2]==b[2] & a[3]==b[3] & a[4]==b[4]) break
233 ^ }
234 ^ }

```

An aerial photograph of a large concrete dam with multiple spillways, situated in a valley. A large reservoir is formed upstream of the dam. The surrounding landscape is covered in dense green forest. A road with a guardrail runs along the right side of the reservoir. In the foreground, a smaller body of water and a grassy area are visible.

Thank you